System Analysis and Design

CSE 307

Lecture 06
Agile Modeling and Prototyping

Learning Objectives

- Understand the roots of agile modeling in prototyping and the four main types of prototyping.
- Be able to use prototyping for human information requirements gathering.
- Understand agile modeling and the core practices that differentiate it from other development methodologies.
- Learn the importance of values critical to agile modeling.
- Understand how to improve efficiency for users who are knowledge workers using either structured methods or agile modeling.

Agile Modeling, but First Prototyping

- Agile modeling is a collection of innovative, user-centered approaches to system development
- Prototyping is an information-gathering technique useful in seeking
  - User reactions
  - Suggestions
  - Innovations
  - Revision plans

Major Topics

- Prototyping
- Agile modeling

Prototyping

- Patched-up
- Nonoperational
- First-of-a-series
- Selected features

Patched-Up Prototype

- A system that works but is patched up or patched together
- A working model that has all the features but is inefficient
- Users can interact with the system
- Retrieval and storage of information may be inefficient

Nonoperational Scale Models

- A nonworking scale mode that is set up to test certain aspects of the design
- A nonworking scale model of an information system might be produced when the coding required by the application is too expensive to prototype but when a useful idea of the system can be gained through prototyping of the input and output only.

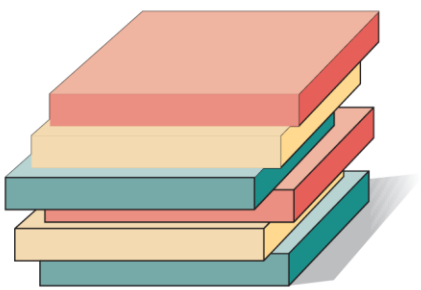
First-of-a-Series Prototype

- Creating a pilot
- Prototype is completely operational
- Useful when many installations of the same information system are planned
- A full-scale prototype is installed in one or two locations first, and if successful, duplicates are installed at all locations based on customer usage patterns and other key factors

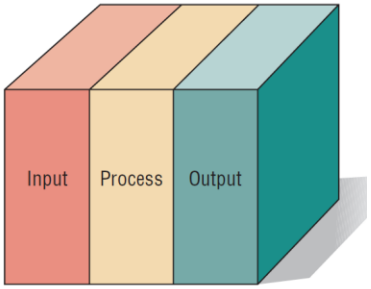
Selected Features Prototype

- Building an operational model that includes some, but not all, of the features that the final system will have
- Some, but not all, essential features are included
- Built in modules
- Part of the actual system

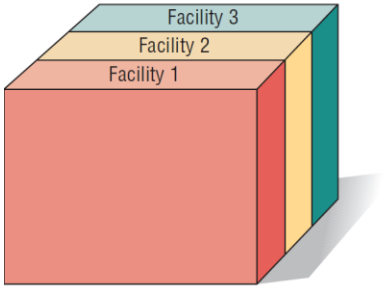
Four Kinds of Prototypes Clockwise, Starting from the Upper Left (Figure 6.1)



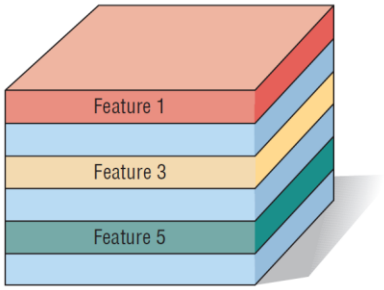
Patched-Up Prototype



Nonoperational Prototype



First-of-a-Series Prototype



Selected Features Prototype

Prototyping as an Alternative to the Systems Life Cycle

- Two main problems with the SDLC
  - Extended time required to go through the development life cycle
  - User requirements change over time
- Rather than using prototyping to replace the SDLC use prototyping as a part of the SDLC

Drawbacks to Supplanting the SDLC With Prototyping

- Drawbacks include prematurely shaping a system before the problem or opportunity is thoroughly understood
- Using prototyping as an alternative may result in producing a system that is accepted by specific groups of users but is inadequate for overall system needs

Guidelines for Developing a Prototype

- Work in manageable modules
- Build the prototype rapidly
- Modify the prototype in successive iterations
- Stress the user interface

Work in Manageable Modules

- It is imperative that an analyst work in manageable modules
- One distinct advantage of prototyping is that it is not necessary or desirable to build an entire working system for prototype purposes
- A manageable module allows users to interact with its key features but can be built separately from other system modules
- Module features that are deemed less important are purposely left out of the initial prototype

Build the Prototype Rapidly

- Speed is essential for successful prototyping
- Analysts can use prototyping to shorten this gap by using traditional information-gathering techniques to find information requirements
- Make decisions that bring forth a working model
- Putting together an operational prototype rapidly and early in the SDLC allows an analyst to gain insight about the remainder of the project
- Showing users early in the process how parts of the system actually perform guards against overcommitting resources to a project that may eventually become unworkable

Modify the Prototype in successive iterations

- Making a prototype modifiable means creating it in modules that are not highly interdependent
- The prototype is usually modified several times
- Changes should move the system closer to what users say is important
- Each modification is followed by an evaluation by users

Stress the User Interface

- Use the prototype is to get users to further articulate their information requirements
- They should be able to see how the prototype will enable them to accomplish their tasks
- The user interface must be well developed enough to enable users to pick up the system quickly
- Online, interactive systems using GUI interfaces are ideally suited to prototypes

Disadvantages of Prototyping

- It can be difficult to manage prototyping as a project in the larger systems effort
- Users and analysts may adopt a prototype as a completed system

Advantages of Prototyping

- Potential for changing the system early in its development
- Opportunity to stop development on a system that is not working
- Possibility of developing a system that more closely addresses users' needs and expectations

Prototyping Using COTS Software

- Sometimes the quickest way to prototype is through the modular installation of COTS software
- Some COTS software is elaborate and expensive, but highly useful

Users' Role in Prototyping

- Honest involvement
  - Experimenting with the prototype
  - Giving open reactions to the prototype
  - Suggesting additions to or deletions from the prototype

Prototype Evaluation Form

(Figure 6.3

Prototype Evaluation Form

Observer Name: Michael Cerveris			Date: 1/06/2013	
System or Project Name: Cloud Computing Data Center		Company or Location: Aquarius Water Filters		
Program Name or Number: Prev. Maint.		Version: 1		
	User 1	**User 2**	**User 3**	**User 4**
User Name	Andy H.	Pam H.		
Period Observed	1/06/2013	1/06/2013		
User Reactions	Generally favorable, got excited about project	Excellent!		
User Suggestions	Add the date when maintenance was performed.	Place a form number on top for reference. Place word WEEKLY in title.		
Innovations				
Revision Plans	Modify on 1/08/2013 Review with Andy and Pam.			

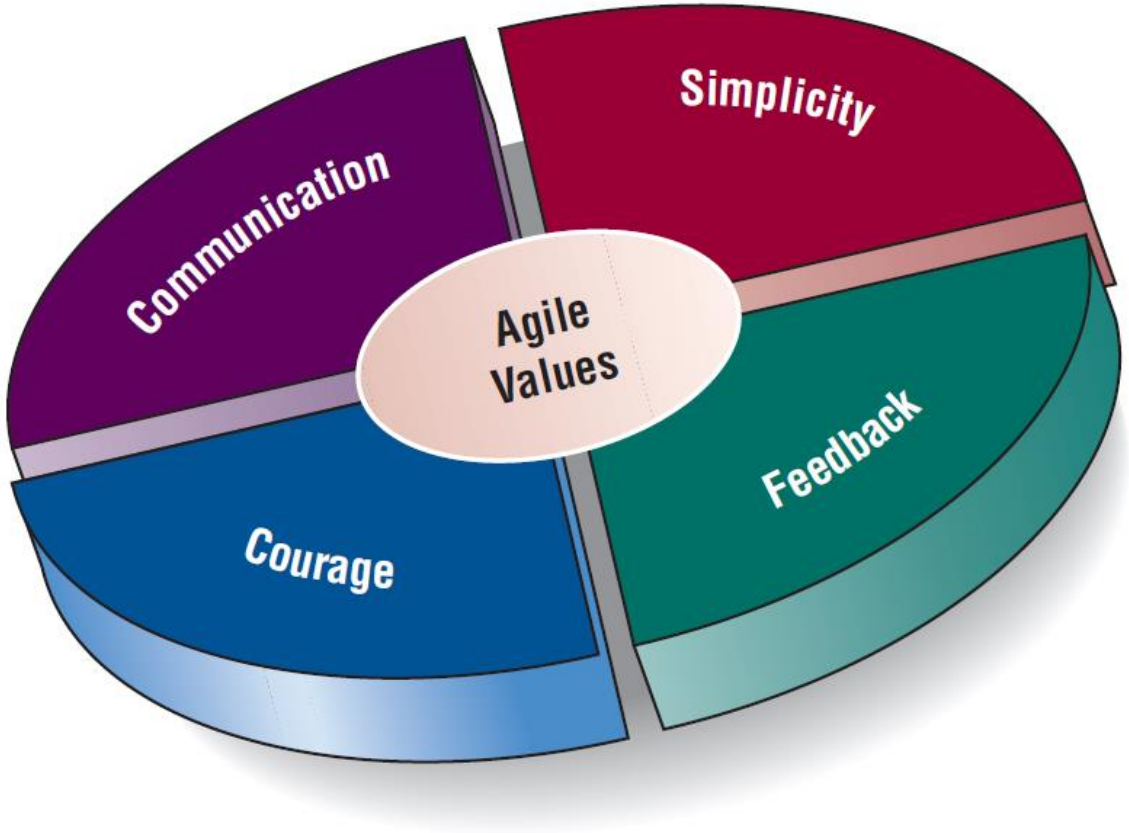
Agile Modeling

- Agile methods are a collection of innovative, user-centered approaches to systems development

Values and Principles of Agile Modeling

- Communication
- Simplicity
- Feedback
- Courage

Values Are Crucial to the Agile Approach (Figure 6.4)



The Basic Principles of Agile Modeling

- Satisfy the customer through delivery of working software
- Embrace change, even if introduced late in development
- Continue to deliver functioning software incrementally and frequently
- Encourage customers and analysts to work together daily
- Trust motivated individuals to get the job done

The Basic Principles of Agile Modeling (continued)

- Promote face-to-face conversation
- Concentrate on getting software to work
- Encourage continuous, regular, and sustainable development
- Adopt agility with attention to mindful design
- Support self-organizing teams

The Basic Principles of Agile Modeling (continued)

- Provide rapid feedback
- Encourage quality
- Review and adjust behavior occasionally
- Adopt simplicity

Four Basic Activities of Agile Modeling

- Coding
- Testing
- Listening
- Designing

Coding

- Coding is the one activity that it is not possible to do without
- The most valuable thing that we receive from code is “learning”
- Code can also be used to communicate ideas that would otherwise remain fuzzy or unshaped

Testing

- Automated testing is critical
- Write tests to check coding, functionality, performance, and conformance
- Use automated tests
- Large libraries of tests exist for most programming languages
- These are updated as necessary during the project
- Testing in the short term gives extreme confidence in what you are building
- Testing in the long term keeps a system alive and allows for changes longer than would be possible if no tests were written or run

Listening

- Listening is done in the extreme
- Developers use active listening to hear their programming partner
- Because there is less reliance on formal, written communication, listening becomes a paramount skill
- A developer also uses active listening with the customer
- Developers assume that they know nothing about the business so they must listen carefully to businesspeople

Designing

- Designing is a way of creating a structure to organize all the logic in the system
- Designing is evolutionary, and so systems are conceptualized as evolving, always being designed
- Good design is often simple
- Design should allow flexibility
- Effective design locates logic near the data on which it will be operating
- Design should be useful to all those who will need it as the development effort proceeds

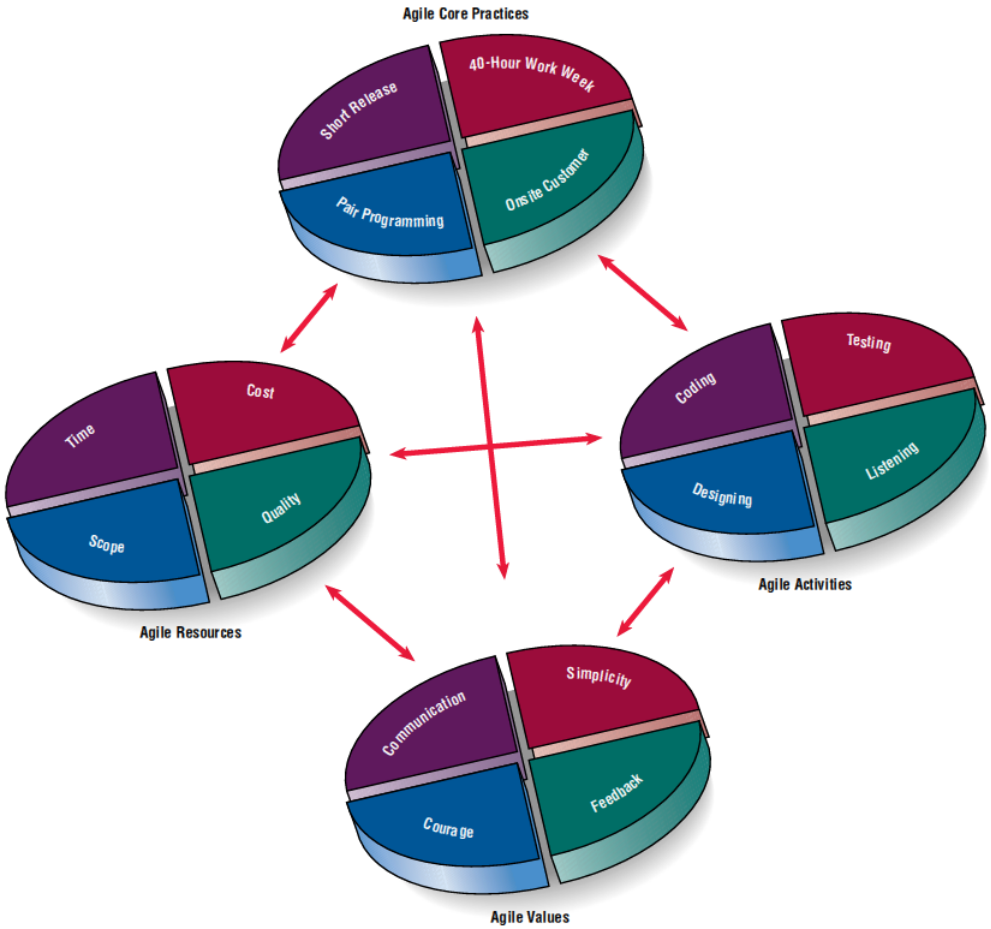
Four Resource Control Variables of Agile Modeling

- Time
- Cost
- Quality
- Scope

Four Core Agile Practices

- Short releases
- 40-hour work week
- Onsite customer
- Pair programming

Agile Core Practices (Figure 6.5)



The Agile Development Process

- Listen for user stories
- Draw a logical workflow model
- Create new user stories based on the logical model
- Develop some display prototypes
- Create a physical data model using feedback from the prototypes and logical workflow diagrams

Writing User Stories

- Spoken interaction between developers and users
- Seeking first and foremost to identify valuable business user requirements
- The goal is prevention of misunderstandings or misinterpretations of user requirements

User Stories Can Be Recorded on Cards (Figure 6.6)

Need or Opportunity:	Apply shortcut methods for faster checkout.					
Story:	If the identity of the customer is known and the delivery address matches, speed up the transaction by accepting the credit card on file and the rest of the customer's preferences such as shipping method.					
		Well Below	Below Average	Average	Above Average	Well Above
Activities:	Coding			✓		
	Testing			✓		
	Listening			✓		
	Designing				✓	
Resources:	Time				✓	
	Cost		✓			
	Quality				✓	
	Scope			✓		

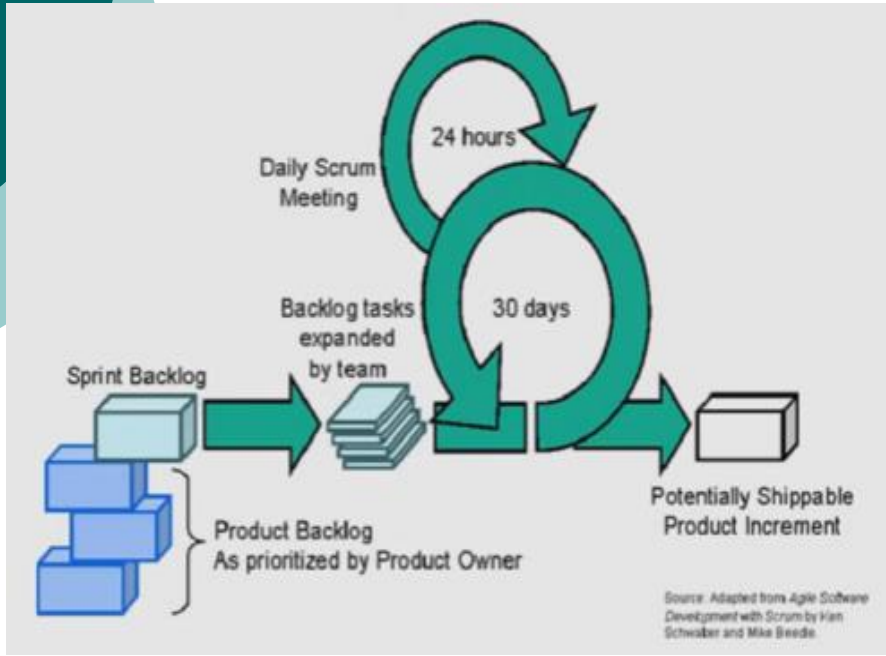
Scrum

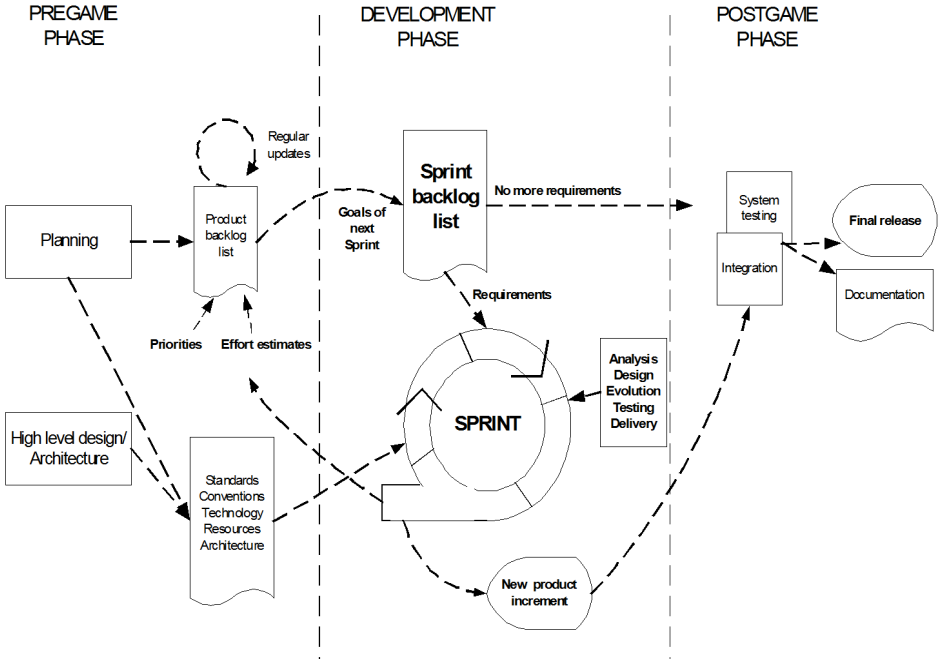
- Begin the project with a high-level plan that can be changed on the fly
- Success of the project is most important
- Individual success is secondary
- Project leader has some (not much) influence on the detail
- Systems team works within a strict time frame

Scrum

- Product backlog
- Sprint backlog
- Sprint
- Daily scrum
- Demo

Scrum Framework





Lessons Learned from Agile Modeling

- Short releases allow the system to evolve
- Pair programming enhances the overall quality
- Onsite customers are mutually beneficial to the business and the agile development team

Lessons Learned from Agile Modeling (continued)

- The 40-hour work week improves worker effectiveness
- Balanced resources and activities support project goals
- Agile values are crucial to success

There Are Six Vital Lessons That Can Be Drawn from the Agile Approach to Systems (Figure 6.7)

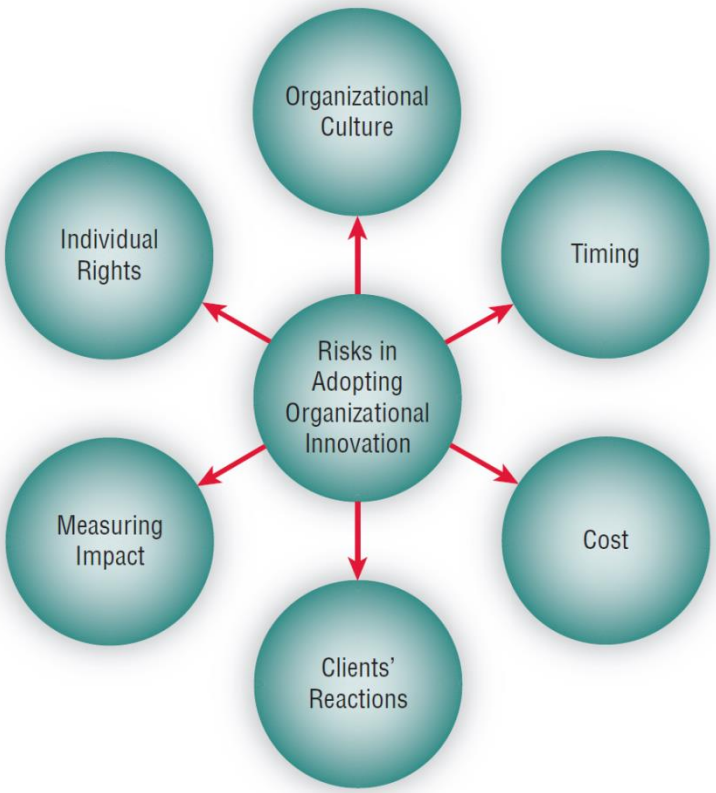
Comparing Agile Modeling and Structured Methods

- Improving the efficiency of systems development
- Risks inherent in organizational innovation

Strategies for Improving Efficiency Can Be Implemented Using Two Different Development Approaches (Figure 6.8)

Strategies for Improving Efficiency in Knowledge Work	Implementation Using Structured Methodologies	Implementation Using Agile Methodologies
Reduce interface time and errors	Adopting organizational standards for coding, naming, etc.; using forms	Adopting pair programming
Reduce process learning time and dual processing losses	Managing when updates are released so the user does not have to learn and use software at the same time	Ad hoc prototyping and rapid development
Reduce time and effort to structure tasks and format outputs	Using CASE tools and diagrams; using code written by other programmers	Encouraging short releases
Reduce nonproductive expansion of work	Managing project; establishing deadlines	Limiting scope in each release
Reduce data and knowledge search and storage time and costs	Using structured data gathering techniques, such as interviews, observation, sampling	Allowing for an onsite customer
Reduce communication and coordination time and costs	Separating projects into smaller tasks; establishing barriers	Timeboxing
Reduce losses from human information overload	Applying filtering techniques to shield analysts and programmers	Sticking to a 40-hour workweek

Adopting New Information Systems Involves Balancing Several Risks (Figure 6.9)



Risks When Adopting New Information Systems

- Fit of development team culture
- Best time to innovate
- Training cost for analysts and programmers
- Client's reaction to new methodology
- Impact of agile methodologies
- Programmers/analysts individual rights

Summary

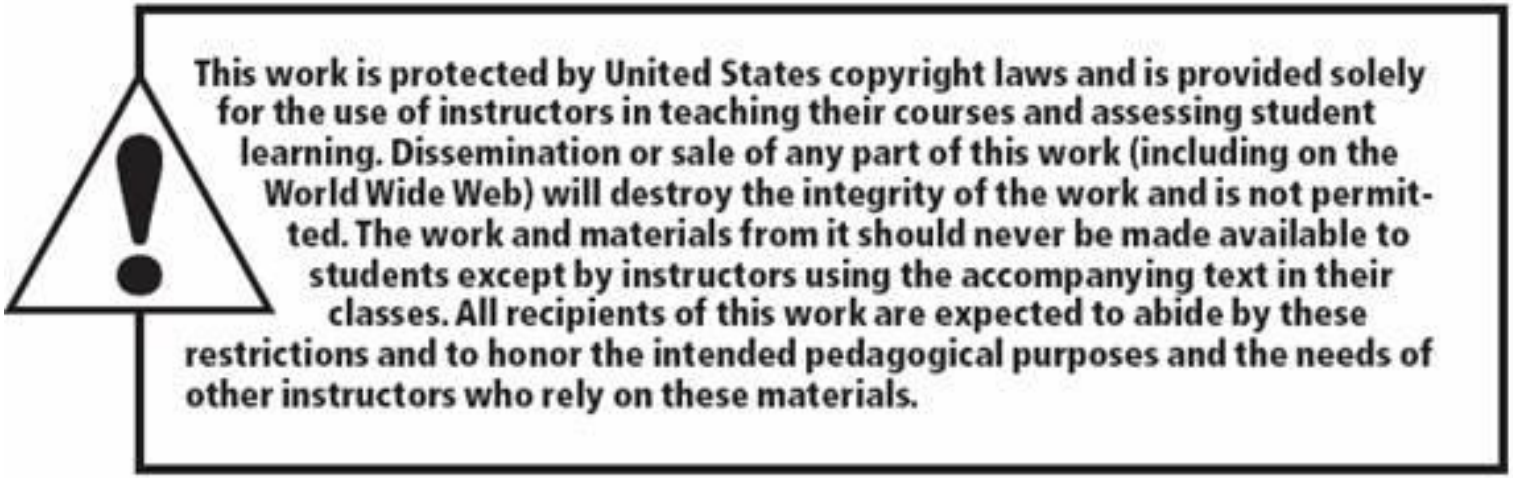
- Prototyping
  - Patched-up system
  - Nonoperational
  - First-of-a-series
  - Selected-features
- Prototype development guidelines
- Prototype disadvantages

Summary (continued)

- Prototype advantages
- Users' role in prototyping
- Agile modeling
- Five values of the agile approach
- Principles of agile development
- Agile activities
- Agile resources

Summary (continued)

- Core practices of the agile approach
- Stages in the agile development process
- User stories
- Agile lessons
- Scrum methodology
- Dangers to adopting innovative approaches



END OF
Lecture 06